Partitioned Hash Join with Duplicates

A Distributed MPI Study

Gabriele Marsili
SPM a.y. 2025/26

1 Introduction

Module 4 moves the partitioned hash join of the previous modules into distributed memory. The algorithm does not change: R and S are partitioned with a shared key-mapping function, each partition is joined on its own, and one collective combines the per-rank results. With the data spread across nodes, the local join now competes with the communication that brings matching partitions onto the same rank. This module measures how those two costs balance, and how the distributed version compares with the shared-memory ones of Modules 2 and 3.

Both implementations reuse the kernel and the input generator of Modules 2 and 3 without changes, so the only difference from the earlier modules is how the work is parallelised. The pure MPI version redistributes partitions with `MPI_Alltoallv`; the hybrid MPI+OpenMP version runs one rank per node and parallelises the local phases across its cores.

2 Distributed Algorithm

2.1 Partitioning and Rank Assignment

The mapping function from Modules 1–3 is unchanged. A 64-bit key k is folded to 32 bits and multiplied by the Fibonacci constant, $p = ((k_{\text{lo}} \oplus k_{\text{hi}}) \cdot \varphi_{32}) \gg (32 - \log_2 P)$. Each partition is assigned to one of the $\mathcal{R}$ MPI ranks by $\text{dest}(p) = p \bmod \mathcal{R}$ with local index $\ell = p \operatorname{div} \mathcal{R}$ in $[0, P/\mathcal{R})$. The constraint $P \bmod \mathcal{R} = 0$ keeps the number of partitions per rank exactly balanced.

2.2 Pipeline

Each rank generates only its slice of R and S with `splitmix64` skip-ahead: no node ever holds the whole relation, there is no initial scatter, and the concatenation of the local slices is byte-for-byte the sequential generator output. The distributed pipeline has eight measured phases:

- *histogram_local* – count the local records of R and S into their destination buckets;
- *scatter_local* – write the records into the send buffer in dest-major order, so each rank's block is contiguous per destination;
- *comm_sizes* – `MPI_Alltoall` of the per-rank counts so the receive buffers can be sized exactly;
- *comm_payload* – `MPI_Alltoallv` of the records;
- *histogram_post* – re-count the received buffer by local partition;
- *scatter_post* – re-lay the received buffer into per-partition order, so the Module 3 kernel is reused unchanged;
- *join_local* – build and probe over the locally owned partitions;
- *reduce_final* – `MPI_Allreduce` of the three aggregates $(\textit{join\_count}, \textit{ck}_1, \textit{ck}_2)$, a tree reduction in $O(\log \mathcal{R})$ rounds.

Each communication phase is preceded by an `MPI_Barrier`, and the timer starts only after it. The barrier lines up all ranks before the collective, so the measured time is the cost of the exchange itself and not the wait for a rank still finishing the previous phase. The local phases between two barriers are timed one after another. Each phase time is then reduced with a maximum across ranks, so the breakdown reports the slowest rank for that phase.

2.3 Choice of the Redistribution Primitive

The redistribution uses `MPI_Alltoallv`. A non-blocking `Isend/Irecv` scheme could overlap the exchange with the work that follows it — *histogram_post*, *scatter_post* and the join — by starting on a partition as soon as its records have arrived from every sender. The breakdown (§4.4) shows how little there is to gain: on four nodes the post-exchange work is about 100 ms in pure MPI against a 1.3 s wall time (∼7%), and about 120 ms of 0.61 s for the hybrid (∼19%). Even that small gain would require splitting the single collective into per-partition non-blocking exchanges, which lose the internal scheduling of the tuned all-to-all and give part of it back before it is realised. A bounded, uncertain gain against a certain restructuring of the redistribution: I kept the blocking collective and treat the overlap as possible future work (§6).

3 Validation

Validation checks the triple $(\textit{join\_count}, \textit{ck}_1, \textit{ck}_2)$ at small, medium and large input sizes. On the small inputs ($N_R, N_S \leq 500$) the result is also checked against a naive join that scans every R–S pair in $O(N_R N_S)$, counting the matches and accumulating the same checksums directly, with no hash table and no partitioning. Since

this naive join shares no code with the kernel, when the two produce the same triple the kernel is almost certainly correct: an independent implementation would not reproduce the same bug. For every size the parallel triple is then compared against the sequential reference `hashjoin_seq`. The medium and large inputs are checked across the whole rank range $\mathcal{R} \in \{1, 2, 4, 8, 32, 64, 128\}$. The small inputs run with only 16 partitions, enough for a few hundred records, and since each rank must own at least one partition ($P \geq \mathcal{R}$) they are limited to $\mathcal{R} \in \{1, 2, 4, 8\}$. The sequential binary accepts `-skew` and `-hot`, so the same checks cover the skewed workload. All 46 configurations passed, reproducible with `scripts/validate_cluster.sh`.

4 Experimental Evaluation

4.1 Setup

Strong-scaling, weak-scaling and breakdown runs use $P = 256$ partitions, seed 42 and $max\_key = N_R/2$, with five repetitions per cell and the median time reported. Strong scaling fixes the global input at $N_R = 50\,\text{M}$ and $N_S = 100\,\text{M}$ records. Weak scaling holds the per-rank workload constant at $N_R/\mathcal{R} = 2\,\text{M}$ records (with $N_S/\mathcal{R} = 4\,\text{M}$). Pure MPI uses one rank per logical CPU (32 ranks per node). The hybrid uses one rank per node with `OMP_NUM_THREADS=32`, `OMP_PROC_BIND=close` and `OMP_PLACES=cores`. The sequential baseline used for speedup is `hashjoin_seq` recompiled with `-march=ivybridge` and timed on a dedicated compute node: the compute nodes do not support the AVX2 that `-march=native` emits on the login node. Speedup is computed against two baselines, one per workload: $t_{\text{seq}}^{\text{unif}} = 4.48\,\text{s}$ on uniform input, $t_{\text{seq}}^{\text{skew}} = 2.30\,\text{s}$ on the skewed input ($\rho = 0.9$, four hot keys), both at $N_R = 50\,\text{M}$, $N_S = 100\,\text{M}$. The skewed baseline is the lower of the two because of memory locality, not output size: with 90% of the records falling in four hot partitions (each holding $\rho/h + (1-\rho)/P \approx 22.5\%$ of the input), the scatter writes into a few near-contiguous streams instead of 256 scattered ones, and the hot hash tables hold few distinct keys and stay cache-resident. In the sequential breakdown this shows up mostly in the scatter phases, which drop by about 3.4×, and to a lesser extent in the join.

4.2 Strong Scaling

Figure 1 plots the speedup against node count. The hybrid is faster than pure MPI from two nodes onwards on both workloads. On uniform input the hybrid speedup climbs steadily from 2.8× at one node to 12.0× at eight. Pure MPI peaks at 7.6× on two nodes (64 ranks) and recovers to 7.1× on eight (256 ranks), but on four nodes (128 ranks) it collapses to 3.0×. The dip is entirely in the payload exchange (`MPI_Alltoallv`): every other phase is stable, but at four nodes this one is slow and erratic, ranging from 0.6 s to 1.5 s across repetitions, while at one or two nodes it is steady. What drives it is the rank count, not the node count: the same 128 ranks spread over eight nodes (16 per node) show the same slow, noisy exchange. Forcing Open MPI's basic linear algorithm in place of the tuned one, on a fixed node set, makes the exchange stable but uniformly slow, so the run-to-run variance comes from the algorithm the library selects in this regime, while the cost itself is the 128-way fan-out. At eight nodes pure MPI uses 256 ranks, so each rank sends half as much data: the exchange settles to about 0.5 s and stops fluctuating, and the speedup recovers. Table 1 gives the per-phase medians: every local phase scales 7–8× from one to eight nodes. Of the three communication phases only *comm_payload* matters, and it anti-scales, peaking at four nodes. *comm_sizes* and *reduce_final* grow too but stay in the milliseconds.

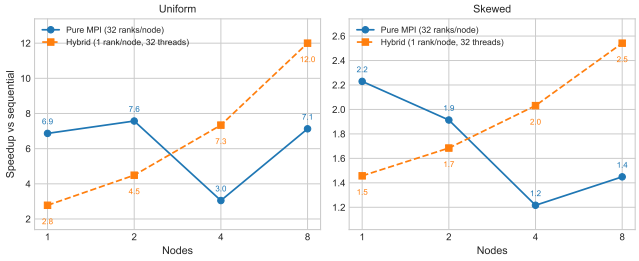

Figure 1: Strong scaling (speedup vs. the sequential baseline) on 1, 2, 4, 8 nodes; note the independent y-axes. $N_R = 50\,\text{M}$, $N_S = 100\,\text{M}$, $P = 256$, five repetitions, median reported.

Table 1: Pure MPI per-phase median times [s], uniform strong scaling ($32N$ ranks). *comm_payload* at four nodes is the erratic phase discussed above; the separate breakdown run of Figure 3 caught 1.13 s for it.

Phase	N=1	N=2	N=4	N=8	$S_{1\to8}$
histogram_local	0.044	0.023	0.011	0.006	7.6×
scatter_local	0.130	0.065	0.033	0.017	7.5×
comm_sizes	<1 ms	11 ms	10 ms	13 ms	—
comm_payload	0.092	0.293	1.35	0.532	0.17×
histogram_post	0.041	0.021	0.013	0.005	7.7×
scatter_post	0.068	0.035	0.022	0.010	6.9×
join_local	0.239	0.121	0.063	0.032	7.4×
reduce_final	<1 ms	2 ms	5 ms	6 ms	—
total	0.65	0.59	1.47	0.63	1.04×

Against the skewed baseline of 2.30 s pure MPI and the hybrid move further apart. Pure MPI never gains from more nodes: it starts at 2.2× on one node, falls to 1.2× at four (the 128-rank regime of the uniform case, made worse by the skew) and climbs back only to 1.4× at eight. Adding ranks enlarges the `MPI_Alltoallv` fan-out, but the hot partitions still land on a few receivers that hold up the whole collective, no matter how many nodes are used. The hybrid keeps improving, from 1.5× on one node to 2.5× on eight, because with at most eight participants the fan-out stays small and each rank has enough work that the limit is load imbalance, not communication. The speedups are also lower simply because the skewed baseline is already 1.9× faster than the uniform one, so there is less time to save.

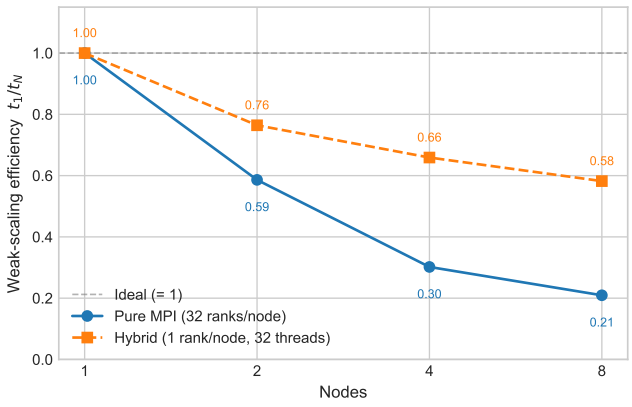

Figure 2: Weak-scaling efficiency t_1/t_N. Per-rank workload constant at 2 M × 4 M records; $P = 256$.

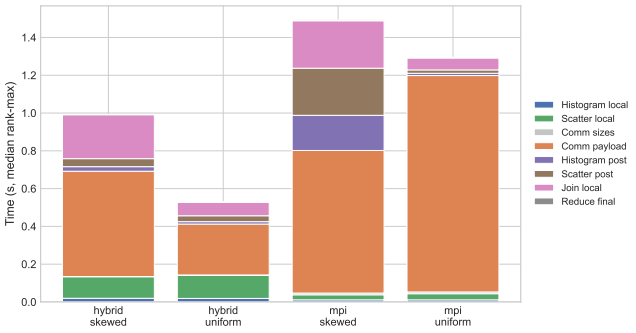

Figure 3: Median rank-max breakdown for the four configurations on four nodes (five repetitions, $N_R = 50$ M, $N_S = 100$ M, $P = 256$).

4.3 Weak Scaling

Figure 2 reports the weak-scaling efficiency t_1/t_N on uniform input, with a per-rank workload of 2 M records of R against 4 M of S. The hybrid scales much better than pure MPI: at eight nodes it keeps 0.58 efficiency, while pure MPI drops to 0.21. The difference comes from how many ranks take part in the collective. With one rank per node the `MPI_Alltoallv` grows from 1 to 8 ranks, and its median time rises only from 15 ms to 52 ms even though the global volume grows 8×. With 32 ranks per node the rank count goes from 32 to 256, and the median payload time rises from 0.12 s to 3.17 s. At one node every transfer is a memory copy, so the network comparison starts at two nodes. From two to eight nodes the hybrid exchange grows by 1.5× while pure MPI grows by 4.7×, tracking its rank count (4×). With the Hockney latency–bandwidth model $\alpha + \beta m$, a rank in an all-to-all posts $\mathcal{R} - 1$ messages of size $V/\mathcal{R}$ (V the fixed per-rank volume), for a per-rank cost of about $(\mathcal{R} - 1)\,\alpha + \beta V$: the bandwidth term is constant by construction in weak scaling, so what grows is the number of message start-ups, linear in $\mathcal{R}$. The hybrid keeps $\mathcal{R}$ equal to the node count and stays bandwidth-bound. Pure MPI multiplies it by 32 and becomes start-up-bound.

Weak scaling was run on uniform input only. Under skew the fixed hot-partition ownership would dominate and grow with N, so a per-rank-fixed setup would measure that imbalance rather than the communication cost it is meant to isolate.

4.4 Phase Breakdown

Figure 3 stacks the eight measured phases for the four configurations on four nodes. The bars also compare the two implementations directly on the same workload: hybrid uniform takes about 0.61 s and pure MPI uniform about 1.30 s, with almost all the gap in *comm_payload*. Pure MPI on uniform spends 88% of the time inside the 128-rank `MPI_Alltoallv`, and the four local phases add up to only about 0.07 s, since each rank handles just a 1/128 slice of the input. For the hybrid it is the other way around: the local phases (now OpenMP-parallel) take a few tens of milliseconds each, and *comm_payload* is the largest part at 44%, even though its eight-rank collective is far cheaper in absolute terms than the 128-rank one. On skewed input pure MPI spends about half its time in communication (50%) and most of the rest in the post-exchange phases, because a few ranks receive most of the records and their re-histogram, re-scatter and join grow accordingly. The hybrid is at its worst on skewed in the local join: the four hot keys fall in four partitions, so up to all four ranks receive one — but those four partitions hold 90% of the records while the other 252 share the rest. Inside such a rank the hot partition is a single iteration of the `schedule(dynamic)` join loop, so one thread grinds through it while the others sit idle, and the join time is set by that one thread.

4.5 Skewed Workload

Figure 4 compares the total time on the two workloads across node counts. Under skew the hybrid slows down only a little, by up to about 0.5 s at eight nodes. Pure MPI does worse: its skewed time does not fall with N but peaks at four nodes and stays well above the one-node time. The distributed setting struggles where Module 3 did not because the imbalance can no longer be balanced away. On a single shared-memory node the `dynamic` schedule let any idle thread pick up a heavy partition, since every thread sees every partition in the same address space. After the all-to-all each rank physically owns its partitions in private memory, so a rank that receives the hot partitions cannot hand them off: no other rank holds those records. This is the usual problem of a partitioned-state farm with hashing, where hashing balances the *number* of partitions across ranks but not their *volume*. A few heavy partitions pile up on a handful of receivers, and the global step waits for the most-loaded one. A way to relieve this is to single out the heavy partitions and, instead of mapping each to one rank, replicate its build side across a few ranks and split its probe side among them, so the heavy join is shared and per-rank work stays near the average. The cost is more communication: replicating the build side puts the same records on several ranks, so part of the join time it saves comes back as extra traffic and memory, and the single `MPI_Alltoallv` becomes a two-path redistribution, one route for the regular partitions and one for the heavy ones. I did not implement it. The net gain is uncertain without measuring, since whether the saved join time

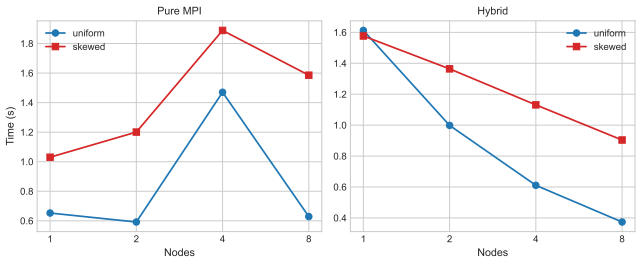

Figure 4: Total time under uniform vs. skewed input ($\rho = 0.9$, 4 hot keys).

Table 2: Cross-module comparison, uniform workload. Speedup over the 4.48 s sequential baseline.

Implementation	Time [s]	Speedup
Sequential	4.48	1.0×
M2 C++ threads (32t, 1 node)	0.53	8.4×
M3 OpenMP loop (32t, 1 node)	0.44	10.1×
M4 pure MPI (1 node, 32 ranks)	0.65	6.9×
M4 pure MPI (2 nodes, 64 ranks)	0.59	7.6×
M4 hybrid (8 nodes, 256 cores)	0.37	12.0×

outweighs the added communication depends on how concentrated the skew is. It would also mean reworking the redistribution, which I deliberately kept simple.

5 Discussion

The sequential baseline runs in 4.48 s, so on a single node both implementations are already well ahead of it: pure MPI with 32 ranks finishes in 0.65 s (6.9× the baseline) and the hybrid with one rank and 32 threads in 1.61 s (2.8×). Between the two, pure MPI is the faster at one and two nodes (0.59 s vs. 1.00 s at two), but from four nodes onwards the hybrid is faster, on both absolute time (0.61 s vs. 1.47 s) and run-to-run variance. The cross-over is driven by the `MPI_Alltoallv` cost: pure MPI keeps the local phases cheap by splitting them across 32 ranks per node, but its communication grows with the total rank count, while the hybrid pays a little more on the local phases and in return runs a much cheaper collective.

5.1 Comparison with previous modules

Figure 5 and Tables 2–3 compare Module 4 with the shared-memory joins of Modules 2 and 3, as speedup over the same sequential baselines (4.48 s uniform, 2.30 s skewed).

On uniform input (Table 2) the shared-memory modules are already strong on one node: M2's C++ threads reach 8.4× and M3's OpenMP loop 10.1×. Both saturate the 32 cores at 8–10× rather than 32×, the sub-linear ceiling expected of a hash join, whose random probes into the hash tables make it memory-bound: the 32 threads share one node's memory bandwidth. Pure MPI at one node is slower (6.9×) because it pays the redistribution it needs for the distributed case even when all data is local: the two extra post-exchange passes and an intra-node `MPI_Alltoallv`, a fixed overhead added on top of the same parallel work. Going past one node only shrinks the local join, which is already cheap; the collective, the one cost that does not shrink, instead grows with the rank count, so by Amdahl's argument it caps the speedup. This is why the hybrid at eight nodes (12.0×) barely beats M3 on one (10.1×): eight times the hardware for about 1.2× more speedup. On uniform data, where the local join is cheap, distribution buys little.

Table 3: Cross-module comparison, skewed workload ($\rho = 0.9$, four hot keys). Speedup over the 2.30 s sequential baseline; Module 2 has no skewed generator.

Implementation	Time [s]	Speedup
Sequential	2.30	1.0×
M3 OpenMP loop (16t, 1 node)	0.37	6.2×
M4 pure MPI (1 node, 32 ranks)	1.03	2.2×
M4 hybrid (8 nodes, 256 cores)	0.90	2.5×

On skewed input (Table 3) the gap reverses, and the reason is load balancing. Hashing balances the *number* of partitions per worker but not their *volume*, so the four hot partitions carry most of the work. M3's OpenMP loop reaches 6.2× on one node because its dynamic schedule is free to balance that volume: any idle thread can drain a hot partition, since all threads share the same memory. Module 4 cannot, because the partitions are pinned to fixed ranks in private memory: no rank can offload its hot partition without moving data, so the most-loaded rank sets the time and even the hybrid at eight nodes (2.5×) stays well below M3 on one. The same imbalance that dynamic scheduling hides in shared memory becomes a hard floor under distribution, and only the partition-aware remapping discussed for the skewed workload would remove it.

6 Hybrid MPI+OpenMP

The hybrid configuration uses one MPI rank per node with `OMP_NUM_THREADS=32`, `OMP_PROC_BIND=close` and `OMP_PLACES=cores`. *Every* per-rank phase is OpenMP-parallel: *histogram_local* and *histogram_post* build per-thread private histograms and merge them at the end, *scatter_local* and *scatter_post* reuse those per-thread tables to derive per-thread offsets so writes never collide on the same slot of the output buffer, and *join_local* runs a `parallel for` with `schedule(dynamic)` and a sum reduction. The static scheduling on the scatter passes mirrors the histogram pass that fed it, so the same input range is visited by the same thread in both phases. The dynamic schedule on the join follows Module 3's loop variant: partition cardinalities vary, and a static split would leave the last threads stuck on the largest partitions. MPI is initialised with `MPI_THREAD_FUNNELED`: several threads may run, but only the main one issues MPI calls. In this code every collective is called outside the OpenMP regions, by the main thread alone, so that level is enough. The stronger `MPI_THREAD_MULTIPLE`, which would let any thread call MPI at the same time, makes the library guard its internal state with locks on every call to stay thread-safe, an overhead this code has no use

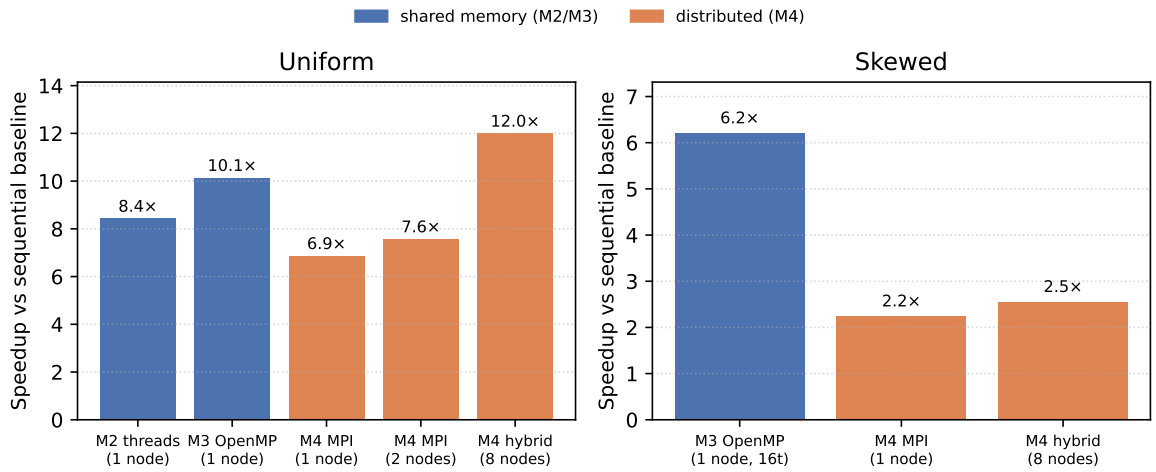

Figure 5: Cross-module speedup over the common sequential baseline (4.48 s uniform, 2.30 s skewed).

for.

Effect on the single-node case. A first iteration of the hybrid only parallelised *join_local* and left the four other local phases single-threaded. It ran in 6.25 s on one node, slower than the sequential baseline, because the only rank had to push 150 M records through one CPU before reaching the parallel join. Parallelising every local phase brings the one-node hybrid down to 1.61 s, 3.9× faster, and is what makes the multi-node curve of Figure 1 monotonic. The first version was Amdahl-bound: the histogram and scatter passes were still serial, and that fraction alone held the one-node time above the sequential baseline no matter how many threads worked the join. The 3.9× came from parallelising those passes, not the join.

Collective cost. Dropping from 256 ranks (pure MPI on 8 nodes) to 8 ranks (hybrid on 8 nodes) shrinks the median `MPI_Alltoallv` on uniform input from 0.53 s to 0.18 s, a 3× gain at the same total payload. `MPI_Allreduce` on the three aggregates stays under a millisecond regardless of node count because it touches only 1 to 8 ranks.

Conclusions

The distributed version keeps the algorithmic structure of the previous modules and reuses the Module 3 kernel unchanged. Within the eight-node budget the hybrid pipeline reaches 12.0× over the sequential baseline on uniform input, with 0.58 weak-scaling efficiency, against 7.6× / 0.21 for pure MPI. The breakdown points the next bottleneck: further speedup is communication-bound, with `MPI_Alltoallv` the largest contribution to wall time in every multi-node configuration. The hybrid pays a non-trivial single-node cost to parallelise the local phases, but recovers it from two nodes onwards.

Set against the shared-memory modules, the gains are real but bounded: on uniform data eight nodes barely exceed Module 3 on one, and on skewed data Module 3's single-node dynamic scheduling stays ahead. The real justification for distribution is capacity rather than speed, spreading an input too large for one node, which this 1.2 GB workload does not exercise.